UNIT:-01 Minimization Technique

→ Minimization of Boolean function using k-map.

- The k-map method is a systematic approach for simplifying a Boolean Expression.
- The basis of k-map method is graphical representation of minterms or maxterms in a chart called Karnaugh map (k-map). k-map contains cells.
- Each cell represents one of the 2^n possible product cells that can be formed from n variables.
- Thus, 2-Variable k-map has 4 cells, 3-Variable k-map has 8 cells and 4-Variable k-map has 16 cells.
- product terms are assigned to the cells of a k-map by labelling each row and each column of the map with a variable, with its complements or with a combination of variables & complements. Fig. depicts the 2-Variable, 3-Variable & 4-Variable maps.

A \ B	$\bar{B}$ 0	B 1
$\bar{A}$ 0	$\bar{A}\bar{B}_0$	$\bar{A}B_1$
A 1	$A\bar{B}_2$	AB_3

(a) 2-Variable k-map

A \ BC	$\bar{B}\bar{C}$ 00	$\bar{B}C$ 01	BC 11	$B\bar{C}$ 10
$\bar{A}$ 0	$\bar{A}\bar{B}\bar{C}_0$	$\bar{A}\bar{B}C_1$	$\bar{A}BC_3$	$\bar{A}B\bar{C}_2$
A 1	$A\bar{B}\bar{C}_4$	$A\bar{B}C_5$	ABC_7	$AB\bar{C}_6$

(b) 3-Variable k-map.

AB \ CD	$\bar{C}\bar{D}$ 00	$\bar{C}D$ 01	CD 11	$C\bar{D}$ 10
$\bar{A}\bar{B}$ 00	$\bar{A}\bar{B}\bar{C}\bar{D}_0$	$\bar{A}\bar{B}\bar{C}D_1$	$\bar{A}\bar{B}CD_3$	$\bar{A}\bar{B}C\bar{D}_2$
$\bar{A}B$ 01	$\bar{A}B\bar{C}\bar{D}_4$	$\bar{A}B\bar{C}D_5$	$\bar{A}BCD_7$	$\bar{A}BC\bar{D}_6$
AB 11	$AB\bar{C}\bar{D}_{12}$	$AB\bar{C}D_{13}$	$ABCD_{15}$	$ABC\bar{D}_{14}$
$A\bar{B}$ 10	$A\bar{B}\bar{C}\bar{D}_8$	$A\bar{B}\bar{C}D_9$	$A\bar{B}CD_{11}$	$A\bar{B}C\bar{D}_{10}$

(c) 4-Variable k-map.

TGSPPU FE ENGG

PDF BY VL

✨ WHY JOIN ?

🏆 Trusted by 80,000+ Engineering Students
🔔 Stay Updated with latest university news
🚀 Make your engineering journey easy with us

Best Regards, Vishwajeet Londhe

© 2026 Vishwajeet Londhe. All Rights Reserved.

Page No.:
Date:
YOUVA

- It is important to note that only one Variable changes, when we move from one cell to another along any row or any column.
- Therefore, the third column and third row in a two Variable K-map have '11' binary representation instead of '10'.
- when two inputs change Simultaneously then digital circuit output goes in a metastable state.
- Output can Swing to either logic '1' or logic '0' State in metastable state.
- This state is to be avoided by prohibiting two inputs from switching Simultaneously.
- We know that any logic function can be represented in SOP and POS form.

TG:SPRUTEENGG

Standard SOP Form on K-map.

⇒ Represent the equation given below on Karnaugh map.

$$Y = \bar{A}\bar{B}\bar{C} + \bar{A}\bar{B}C + A\bar{B}\bar{C} + AB\bar{C} + ABC.$$

Soln:-

The logical expression in standard SOP form can be represented with the help of a K-map by simply entering 1's in the cells of the K-map corresponding to each minterm present in the equation.

The remaining cells are filled with zeros.

A \ BC	$\bar{B}\bar{C}$	$\bar{B}C$	BC	$B\bar{C}$
$\bar{A}$	1 (0)	1 (1)	(3)	(2)
A	1 (4)	(5)	1 (7)	1 (6)

OR

A \ BC	00	01	11	10
0	1 (0)	1 (1)	(3)	(2)
1	1 (4)	(5)	1 (7)	1 (6)

"1"s correspond to the minterms in the given Boolean Expression.

ex. plot the following Boolean Expression on k-map.

$$Y = \bar{A}\bar{B}\bar{C}\bar{D} + \bar{A}\bar{B}CD + \bar{A}BC\bar{D} + AB\bar{C}\bar{D}$$

AB \ CD	$\bar{C}\bar{D}$	$\bar{C}D$	CD	$C\bar{D}$
$\bar{A}\bar{B}$	1 (0)	(1)	1 (3)	(2)
$\bar{A}B$	(4)	(5)	(7)	1 (6)
AB	1 (12)	(13)	(15)	(14)
$A\bar{B}$	(8)	(9)	(11)	(10)

OR

AB \ CD	00	01	11	10
00	1 (0)	(1)	1 (3)	(2)
01	(4)	(5)	(7)	1 (6)
11	1 (12)	(13)	(15)	(14)
10	(8)	(9)	(11)	(10)

TG: SPPU_FE_ENGG

* Summary of All Boolean laws:-

OR law - (1) $A+0=A$
(2) $A+1=1$
(3) $A+A=A$
(4) $A+\bar{A}=1$

AND law - (1) $A.0=0$
(2) $A.1=A$
(3) $A.A=A$
(4) $A.\bar{A}=0$

Commutative law →
(1) $A+B=B+A$
(2) $A.B=B.A$

Associative law →
(1) $(A+B+C)=A+(B+C)$
(2) $(A.B).C=A.(B.C)$

Distributive law →
(1) $A.(B+C)=AB+AC$
(2) $A+BC=(A+B)(A+C)$

Complementation law →
(1) $\bar{0}=1$
(2) $\bar{1}=0$
(3) $\bar{\bar{A}}=A$
(4) $A.\bar{A}=0$
(5) $A+\bar{A}=1$

Absorption law → (1) $A+A.B=A$
(2) $A.(A+B)=A$

Idempotance law → (1) $A+A=A$
(2) $A.A=A$

Identity law → (1) $A.1=A$
(2) $A+1=1$

Involution law → (1) $\bar{\bar{A}}=A$
(2) $A+\bar{A}B=A+B$

TG: SPPU_ENGG

✱ De-Morgan's Theorem:-

1) It states that the complement of a product of Variable is equal to the sum of the complements of the Variables.

$$\overline{(A.B.C)} = \overline{A} + \overline{B} + \overline{C}$$

2) It states that the complement of a Sum of Variable is equal to the product of the complements of the Variables.

$$\overline{(A+B+C)} = \overline{A} . \overline{B} . \overline{C}$$

✱ Min-term & Max-term:-

→ Min-term.

Each of the combination is called a minterm of standard product.

A Min-term is represented by m_i, where i is the decimal Equivalent of the binary number to which the minterm is designed.

e.g. for Boolean function in two Variables. the minterms are –

$m_0 = \overline{x} . \overline{y}$, $m_1 = \overline{x} . y$

$m_2 = x . \overline{y}$, $m_3 = x . y$.

In a Min-term, the binary Variable is unprimed if the Variable is 1 & it is primed if the Variable is 0. i.e. if the Minterm is $x\overline{y}$, then it Means $x = 1$ & $y = 0$.

→ Max-term.

In similar way, if the variables are combined together with OR Operation, then the term Obtained is called Maxterm or standard sum.

TG: GPRU_FE_ENGG

A Max-term is represented by M_i where 'i' is the decimal equivalent of the binary number where the maxterm is designed.

In a Max-term, the binary variable is non-primed the variable is 0 & it is primed if the variable is 1, i.e., if the Max-term is $\bar{x}+y$ then it means $\bar{x}=1$ & $y=0$.

For a Boolean function in two variables, the max-terms are:

$M_0 = x+y$, $M_1 = x+\bar{y}$

$M_2 = \bar{x}+y$, $M_3 = \bar{x}+\bar{y}$

☆ Step to convert SOP to standard SOP Form:-

→ Write down all the terms.

→ If one and more variables are missing in any product term, expand that term by multiplying it with the sum of each one of the missing variable and its complement.

→ Drop the redundant terms:-

e.g. $f(A,B,C) = AB + A\bar{C} + BC$

Find Missing Variables for each & every product term. Apply the OR operator b/w the missing literal & its complement e.g. $(C+\bar{C})$

Now join it with product term through the AND operator.

$$F(A,B,C) = AB(C+\bar{C}) + A\bar{C}(B+\bar{B}) + BC(A+\bar{A})$$
$$= ABC + AB\bar{C} + AB\bar{C} + A\bar{B}\bar{C} + ABC + \bar{A}BC$$
$$= ABC + \bar{A}BC + AB\bar{C} + A\bar{B}\bar{C}$$

TG:SRPUPFENGG

M T W T F S
Page No.:
Date:
YOUVA

☆ Steps to convert POS to standard POS Form:-

→ write down all the terms.
→ If one or more variables are Missing in any sum term, expand that term by adding the product of each one of the Missing term & its complement.
→ Drop out the redundant terms.

e.g. $F(A,B,C) = (A+B).(A+C).(A+\bar{C})$

Find the missing literal for each sum term. Now join the Missing literal, Missing literal with AND operator, & then join this term with the sum term using OR operator.

$$F(A,B,C) = (A+B+C\bar{C}).(A+C+B\bar{B}).(A+\bar{C}+B\bar{B})$$
$$= (A+B+C).(A+B+\bar{C}).(A+C+B).(A+C+\bar{B})$$
$$\cdot (A+\bar{C}+B)$$
$$= (A+B+C).(A+B+\bar{C}).(A+\bar{B}+C).(A+\bar{C}+\bar{B})$$

EX:- If $F(A,B,C) = Em(0,3,5,7)$ with its truth-table & express F in SOP Form & POS Form.

Soln:- we prepare truth-table for F. The function F has three input variables A, B, C.

Truth-table for $F(A,B,C)$. We regard F as Output variable.

To Express $F(A,B,C)$ in standard sop form

$\therefore F(A,B,C) = Em(0,3,5,7)$
$= m_0 + m_3 + m_5 + m_7$ in standard SOP Form.

$\therefore F = \bar{A}\bar{B}\bar{C} + \bar{A}BC + A\bar{B}C + ABC$ To Express $F(A,B,C)$ in standard POS form using De-Morgan's

TG: SPPU_FE_ENGG

law, we take complement of F, i.e. changing 0 to 1 & 1 to 0.

$\therefore \overline{F}(A \cdot B \cdot C) = m_1 + m_2 + m_4 + m_6$

Again taking complement of $\overline{F}$

$$\overline{\overline{F}} = \overline{(m_1 + m_2 + m_4 + m_6)}$$
$$= \overline{m_1} \cdot \overline{m_2} \cdot \overline{m_4} \cdot \overline{m_6}$$

De-morgan's law $= \overline{A+B} = \overline{A} \cdot \overline{B}$ $(\because \overline{m_i} = m_i)$

$\therefore F = m_1 \cdot m_2 \cdot m_4 \cdot m_6$

$\therefore F = M(1,2,4,6)$ in standard POS Form.

Ex = Which of the following expression is equivalent to $Z = A\overline{B} + C$, where A represents MSB & C represents LSB of the binary numbers?

(1) $Z = \Sigma m(0,2,6)$ (2) $Z = \pi M(1,3,4,5,7)$
(3) $Z = \Sigma m(1,3,4,5,7)$ (4) $Z = \pi M(0,2,6)$

Soln:- $Z = A\overline{B} + C$

We convert Z in standard SOP form. The given expression has 3-input variables A, B, C.

$$Z = A\overline{B}(C+\overline{C}) + C(A+\overline{A})(B+\overline{B})$$
$$= A\overline{B}C + A\overline{B}\overline{C} + CAB + CA\overline{B} + C\overline{A}B + C\overline{A}\overline{B}$$
$$= A\overline{B}C + A\overline{B}\overline{C} + ABC + \overline{A}BC + \overline{A}\overline{B}C$$
$(\because C\overline{A}B$ is redundant)
$$= (101, 100, 111, 011, 001)$$
$$= 5, 4, 7, 3, 1.$$
$$= 1, 3, 4, 5, 7.$$

$Z = \Sigma m(1,3,4,5,7)$ is SOP form.

Taking complement Twice,

$Z = \pi M(0,2,6)$ in POS form.

TG SPPU FE ENGG

☆ K-Map Simplification :-

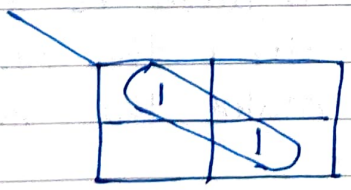
* One K-Map is a method of simplifying Boolean expressions. The required Boolean Results are transformed from a truth-table onto a two-dimensional grid, where each cell represents one combination of input conditions.

* Cells are also known as 'min-terms', while each cell position represents the corresponding output values of Boolean Functions.

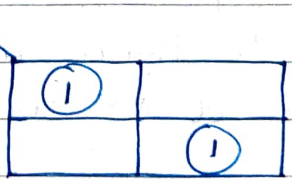
* Optimal groups of '1's or '0's are identified, which represents the terms of a 'canonical form' of the logic in the original truth-table. These terms can be used to write a minimal Boolean Expression representing the required logic.

☆ K-Map Simplification Rules (MOST IMPORTANT):-

1. We can either group 0's with 0's or 1's with 1's but we cannot group 0's & 1's together.
2. Groups may overlap each other.
3. We can only create a group whose number of cells can be represented in the power of 2. In other words, a group can contain 2^n i.e. 1, 2, 4, 8, 16... number of cells.
4. Groups can be either horizontal or vertical. we cannot create group of diagonal or any other shape.



Incorrect

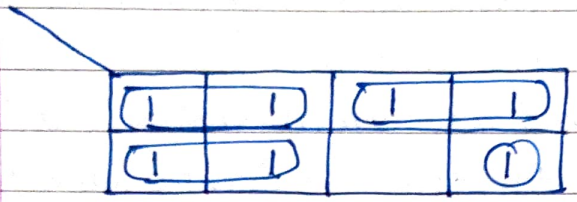


Correct

FG-SPRU_FE_ENGG

M T W T F S S
Page No.:
Date:
YOUVA

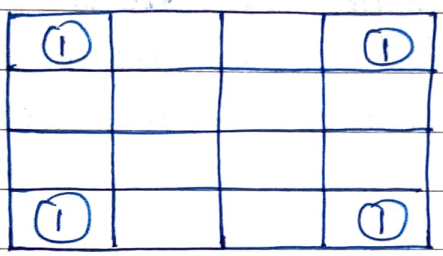
(5) Each group should be as large as possible. Opposite grouping is allowed. This is the example of opposite grouping.



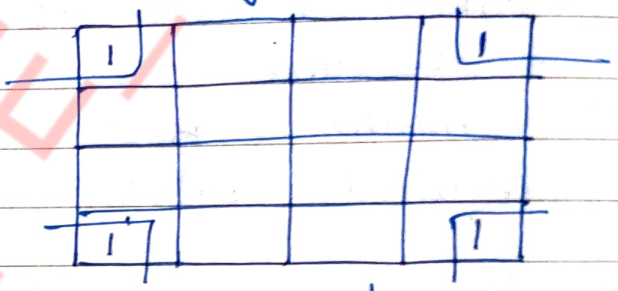
Incorrect

correct

(6) Opposite grouping & corner grouping are allowed.



Incorrect



correct

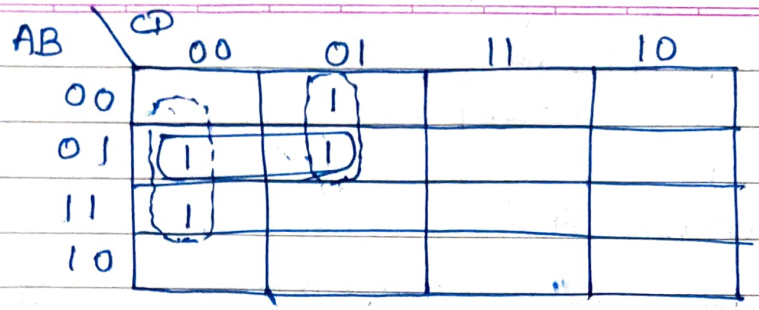
(7) There should be as few groups as possible.

(8) Redundant Group:-

i) A Redundant group is a group whose all 1's are overlapped by other groups. We eliminate these redundant groups to get the Simplified Boolean expression. The elimination of redundant groups results in a simple logic circuit.

ii) In the K-map of Fig. the group which is represented by the square is a redundant group & hence while writing the equations we ignore it or we don't make this kind of group.

TG:SPPU_FE_ENG



(9) Each group should have the largest number of 'ones' a group can't contain an empty cell or cell that contains 0.

(10) We group the number of ones in the decreasing order first, we have to try to make the group of eight, then for four, after that two & lastly 1.

(11) The elements in one group can also be used in different groups only. When the size of group is increased.

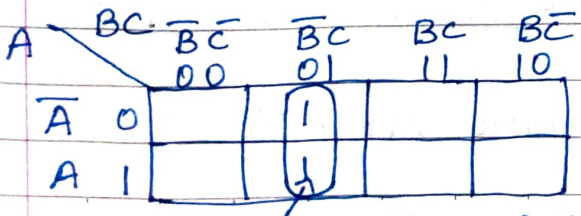
(12) The elements located at the edges of the table are considered to be adjacent, so we can group these elements.

(13) We can consider the 'don't' care condition' only when they did in increasing the group-size. Otherwise 'don't care' elements are discarded.

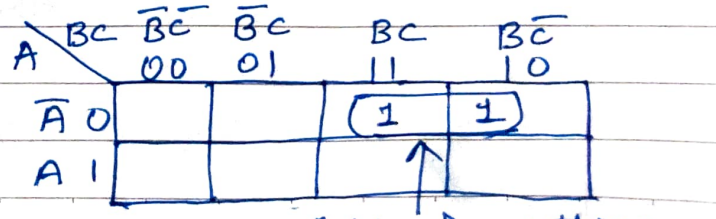
☆ Method of grouping :-

(A) Grouping of two adjacent 1's (pair) :~

A group of two adjacent cell in a k-map is called pair. A pair cancels one variable of Min-term or Max-term in a k-map simplification.

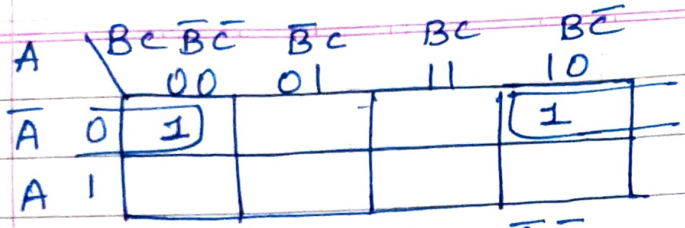


Common Variable from this pair is $\bar{B}C$

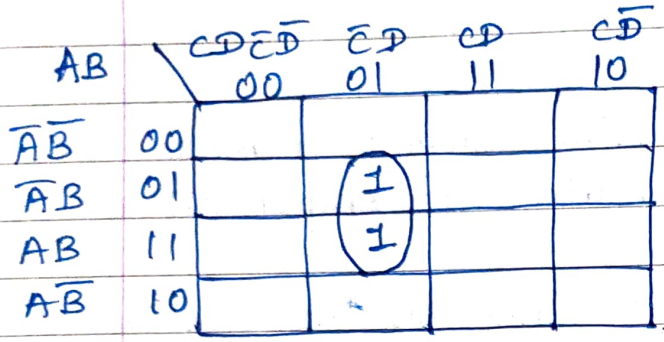


Common Variable from this pair is $\bar{A}B$

TG:SPPU_FE_ENGG



Common Variable $\bar{A}\bar{C}$



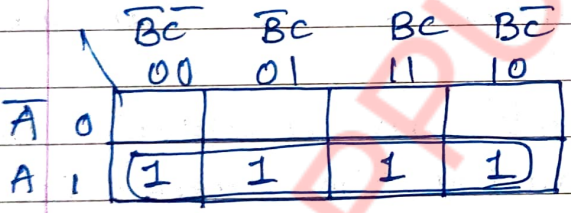
Common Variable - $B\bar{C}D$

Common Variable - $A\bar{B}\bar{D}$

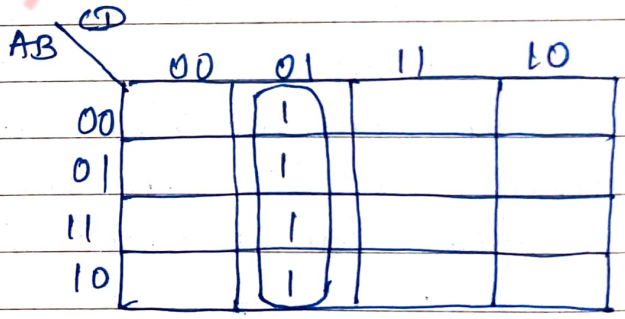
(B) Grouping of Four Adjacent 1's (Quad) :-

A group of Four 1's that are adjacent to one another is called 'quad'.

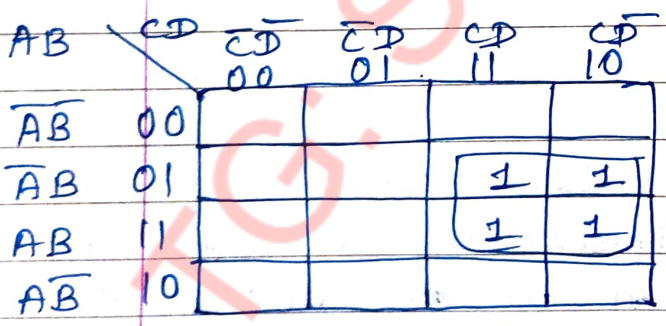
A Quad cancels two variables of Min-term or Max-term in a K-Map Simplification.



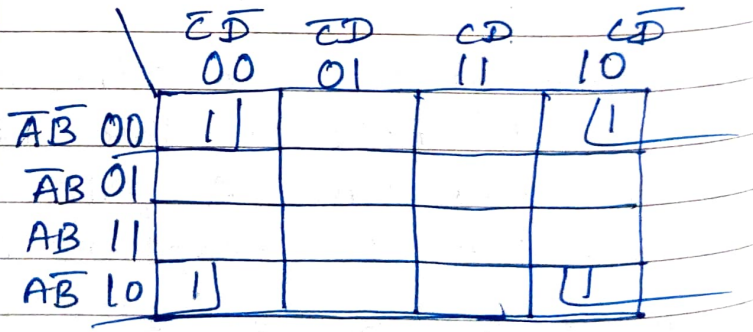
Common Var - A.



Common Var → $\bar{C}D$



Common Var - BC

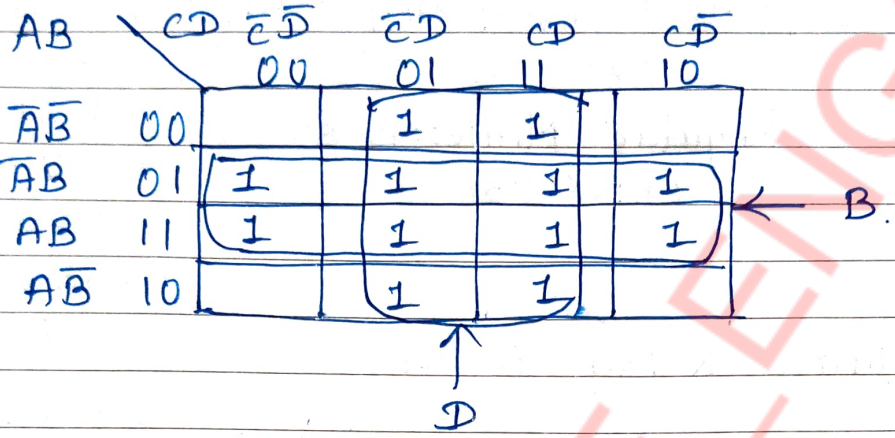


Common var - $B\bar{D}$

KG SPPU FE ENGG

(c) Grouping of Eight adjacent 1's (octet) :~

A group of Eight adjacent cells in a K-Map. An Octet cancels three Variables of Min-term or Max-term in a K-Map Simplification.



(d) Don't care condition :~

(1) The 'Don't care' condition says that can use the blank cells of a K-Map to make a group of the Variables.

(2) To make a group of cells, we can use the 'don't care' cells to make a large group of cells.

(3) The cross (x) symbol is used to represent the 'don't care' cell in K-Map. It is also represented by 'd' in the truth-table.

(4) Don't care terms are important to consider in Minimising using K-Maps & also the Quine-Mc Cluskey Algorithm.

TG.SPUR_ENGG

(Imp - 4 Marks)

EX:- Minimize the following function using K-map and realize using logic gates:

$F(A, B, C, D) = \Sigma m(1,3,7,11,15) + d(0,2,5)$

Soln:-

$Y = F = m_1 + m_3 + m_7 + m_{11} + m_{15} + d[0,2,5]$

Regular Minterms so enter 1's

don't care conditions are treated as 1's so enter X Marks.

→ The required K-Map is shown in figure.

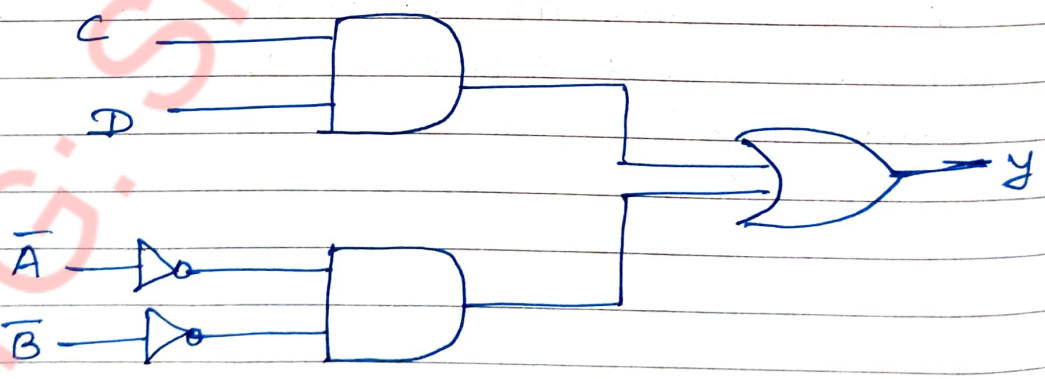
AB \ CD	$\overline{C}\overline{D}$ 00	$\overline{C}D$ 01	CD 11	$C\overline{D}$ 10
$\overline{A}\overline{B}$ 00	X	1	1	X
$\overline{A}B$ 01	0	X	1	0
AB 11	0	0	1	0
$A\overline{B}$ 10	0	0	1	0

Group 2: $\overline{A}\overline{B}$

Don't care conditions are treated as 1's

group 1: CD

→ simplified expression $F = CD + \overline{A}\,\overline{B}$



Implementation using logic gates

TG:SPRUTEENGG

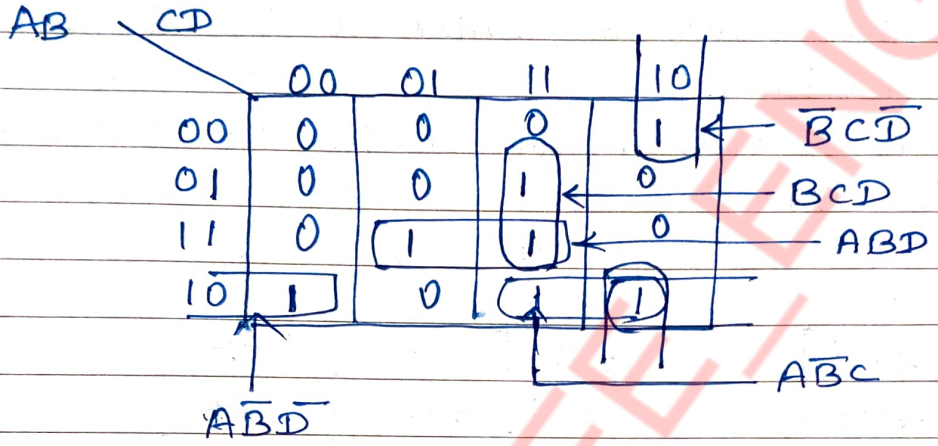
(Imp – 6 Marks)

Ex:- Solve the following Equations using corresponding minimization techniques:

1. $Z = f(A, B, C, D) = \Sigma m(2, 7, 8, 10, 11, 13, 15)$
2. $Z = f(A, B, C, D) = \Sigma m(0, 3, 4, 9, 10, 12, 14)$

Soln:-

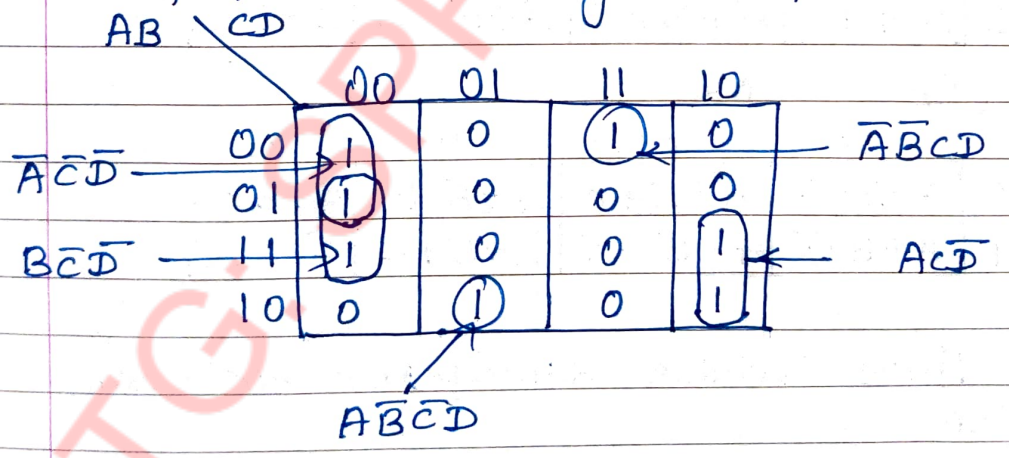
1. $Z = f(A, B, C, D) = \Sigma m(2, 7, 8, 10, 11, 13, 15)$



$\therefore Z = \bar{B}C\bar{D} + BCD + ABD + A\bar{B}C + A\bar{B}\bar{D}$

2. $Z = f(A, B, C, D) = \Sigma m(0, 3, 4, 9, 10, 12, 14)$

→ Simplification using K-Map.



$\therefore Z = \bar{A}\bar{C}\bar{D} + B\bar{C}\bar{D} + AC\bar{D} + \bar{A}\bar{B}CD + A\bar{B}C\bar{D}$

TG-JNTUFE/ENGG

✡ Product of Sum (POS) Simplification:-

K-map Representation of POS Form:-

→ The POS form Equations consist of maxterms.
→ In the k-map corresponding to POS form Equation we have to enter a 0 corresponding to each maxterm.
→ The k-maps corresponding to the POS form are shown in fig.

A \ B	B (0)	$\bar{B}$ (1)
A (0)	A+B	A+$\bar{B}$
$\bar{A}$ (1)	$\bar{A}$+B	$\bar{A}$+$\bar{B}$

B \ A	A (0)	$\bar{A}$ (1)
B (0)	A+B	$\bar{A}$+B
$\bar{B}$ (1)	A+$\bar{B}$	$\bar{A}$+$\bar{B}$

OR

(a) Two Variable K-map for POS form

A \ BC	00	01	11	10
0	A+B+C	A+B+$\bar{C}$	A+$\bar{B}$+$\bar{C}$	A+$\bar{B}$+C
1	$\bar{A}$+B+C	$\bar{A}$+B+$\bar{C}$	$\bar{A}$+$\bar{B}$+$\bar{C}$	$\bar{A}$+$\bar{B}$+C

C \ AB	00	01	11	10
0	A+B+C	A+$\bar{B}$+C	$\bar{A}$+$\bar{B}$+C	$\bar{A}$+B+C
1	A+B+$\bar{C}$	A+$\bar{B}$+$\bar{C}$	$\bar{A}$+$\bar{B}$+$\bar{C}$	$\bar{A}$+B+$\bar{C}$

OR

(b) Three Variable K-map for POS form.

AB \ CD	00	01	11	10
00	A+B+C+D	A+B+C+$\bar{D}$	A+B+$\bar{C}$+$\bar{D}$	A+B+$\bar{C}$+D
01	A+$\bar{B}$+C+D	A+$\bar{B}$+C+$\bar{D}$	A+$\bar{B}$+$\bar{C}$+D	A+$\bar{B}$+$\bar{C}$+$\bar{D}$
11	$\bar{A}$+$\bar{B}$+C+D	$\bar{A}$+$\bar{B}$+C+$\bar{D}$	$\bar{A}$+$\bar{B}$+$\bar{C}$+$\bar{D}$	$\bar{A}$+$\bar{B}$+$\bar{C}$+D
10	$\bar{A}$+B+C+D	$\bar{A}$+B+C+$\bar{D}$	$\bar{A}$+B+$\bar{C}$+$\bar{D}$	$\bar{A}$+B+$\bar{C}$+D

(c) Four variable K-Map for POS form

TG:@GBU_FREE_NOTES

- The entries in the K-map can be shown in terms of the maxterms $M_0, M_1 \ldots$ etc. as shown in Fig.

(a)

A \ B	0	1
0	M_0	M_1
1	M_2	M_3

(b)

A \ BC	00	01	11	10
0	M_0	M_1	M_3	M_2
1	M_4	M_5	M_7	M_6

(c)

AB \ CD	00	01	11	10
00	M_0	M_1	M_3	M_2
01	M_4	M_5	M_7	M_6
11	M_{12}	M_{13}	M_{15}	M_{14}
10	M_8	M_9	M_{11}	M_{10}

K-map in terms of maxterms.

→ Representation of standard POS Form on K-Map:-

→ Logical expressions in the standard pos form can be represented on K-map by entering 0's in the cells of K-map corresponding to each maxterm present in the given equation.

→ The remaining cells are filled with 1's

Ex:- Represent the following standard POS expression on Karnaugh Map.

$$Y = (A+B+C)(A+\overline{B}+C)(\overline{A}+\overline{B}+C).$$

Soln:-

→ Each term in the given logical equation is a Maxterm.

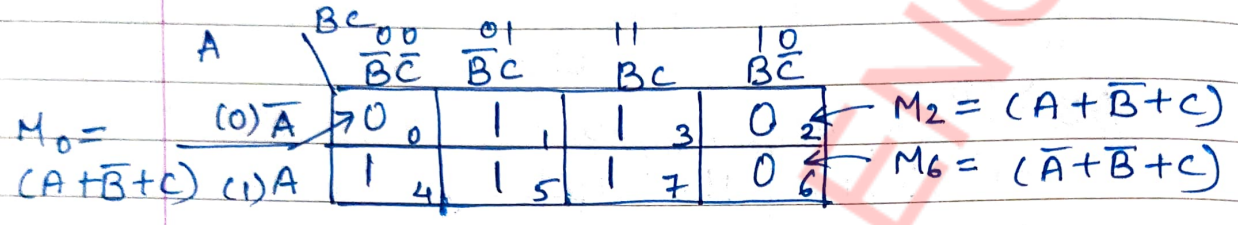
→ Enter a 0 corresponding to each maxterm as shown in Fig.

→ The given expression has three maxterms as follows,

TG: SPPU FE ENGG

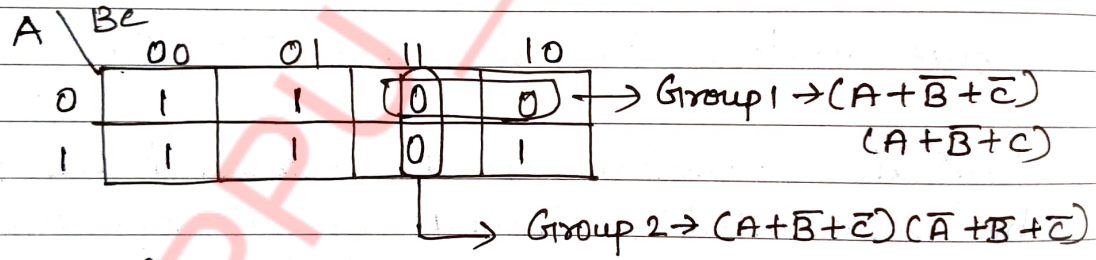
$(A+B+C) = M_0$,
$(A+\bar{B}+C) = M_2$,
$(\bar{A}+\bar{B}+C) = M_6$.

→ Hence we have to write the structure of 3 variable K-map as usual and enter 0's at M_0, M_2 and M_6 as shown in Fig.



Representation of standard POS on K-map.

Ex:- Find the expression in the POS form for the K-map shown in Fig.



Given K-Map.

Soln :-

Simplification:

Group 1 → $(A+\bar{B}+\bar{C})(A+\bar{B}+C)$

$= A.A + A.\bar{B} + A.C + A\bar{B} + \bar{B}\bar{B} + \bar{B}\bar{C} + AC + \bar{B}.C + \bar{C}.C$

But $AA = A$, $\bar{B}.\bar{B} = \bar{B}$, $C.\bar{C} = 0$, $A\bar{B} + A\bar{B} = A\bar{B}$

∴ Group 1 $= A + A\bar{B} + AC + \bar{B} + \bar{B}C + A\bar{C} + \bar{B}\bar{C}$

$= A(1+\bar{B}) + A(C+\bar{C}) + \bar{B}(1+C) + \bar{B}\bar{C}$

$= A + A + \bar{B} + \bar{B}\bar{C}$

$= A + \bar{B}(1+\bar{C})$

Group 1 $= A + \bar{B}$

TG:SPPU_FE_ENGG

M T W T F S S
Page No.:
Date:
YOUVA

Conclusion:-

→ When a pair of 0s is formed, the variable which changes will get Eliminated e.g. C changes for the pair of 0's in group 1. Hence it is Eliminated.

∴ Group 2 → $\bar{B} + \bar{C}$

∴ Final minimized Equation is as follows:

$$Y = (A + \bar{B})(\bar{B} + \bar{C})$$

(1) (2)

Group 2 → $(A + \bar{B} + \bar{C})$ $(\bar{A} + \bar{B} + \bar{C})$

Remain unchanged

Variable that changes its value hence it gets Eliminated

TG: SPPU_FE_ENGG

☆ Introduction to Quine Mc Cluskey Method :-

K-map method is a convinient Method for minimizing Boolean functions up to 4 variables. But it is difficult to simplify the Boolean function having more than 4 variables by using this method.

Quine Mc Cluskey Method is a tabular Method based on the concept of prime Implicants. prime implicant is a product or sum term, which cannot be further reduced by combining with any other product or sum terms of the given Boolean Function.

This tabular Method is useful to get the prime Implicants by repeatedly using the following Boolean Identity.

$$xy + xy' = x(y + y') = x(1) = x$$

☆ Procedure of Quine Mc Cluskey (Tabular Method):-

1. Arrange the given Minterms in an Ascending order & make the group based on the number of 1's present in their binary representation.
2. Compare the Minterm present in successive groups If there is a change in only one bit position, then take the pair of those two minterms. place the symbol '–' in the different bit position & keep the remaining bits as it is.
3. Repeat step ② with newly formed terms. we get all prime Implicants.
4. Formulate the prime implicant table. It consists of set of rows & columns. prime implicants can be placed in row-wise &

TG-SPU FE-ENGG

Minterms can be placed in Column wise place '1' in the cells corresponding to the Minterms that are covered in each prime Implicant.

(5) Find the essential prime implicants by observing each column. If this minterm is covered only by one prime Implicant, then it is essential prime Implicants will be part of the simplified Boolean function.

(6) Reduce the prime Implicant table by removing the repeated row of each essential prime implicant, Repeat step 5 for reduced prime implicant table Stop the process when all Min-terms of given Boolean functions are over.

Ex:- Simplify the following Boolean function -
$f(w,x,y,z) = Em(2,6,8,9,10,11,14,15)$

Soln

The given Boolean function is in sum of min-term form. It is having four variables. w, x, y, z. We first prepare 4-bit binary table.

Number	Binary Representation
0	0 0 0 0
1	0 0 0 1
2	0 0 1 0
3	0 0 1 1
4	0 1 0 0
5	0 1 0 1
6	0 1 1 0
7	0 1 1 1
8	1 0 0 0
9	1 0 0 1
10	1 0 1 0

TG:SPRUFFING

11	1 0 1 1
12	1 1 0 0
13	1 1 0 1
14	1 1 1 0
15	1 1 1 1

Using the above table, we express the given Min-terms in 4-bit binary form.

Min terms	W	X	Y	Z
2	0	0	1	0
6	0	1	1	0
8	1	0	0	0
9	1	0	0	1
10	1	0	1	0
11	1	0	1	1
14	1	1	1	0
15	1	1	1	1

Step-II we arrange the given Min-terms into groups based on 1's present in the binary Equivalents

Group Name	Min-terms	W	X	Y	Z	No. of 1's in each row
GA1	2	0	0	1	0	1
	8	1	0	0	0	
GA2	6	0	1	1	0	2
	9	1	0	0	1	
	10	1	0	1	0	
GA3	11	1	0	1	1	3
	14	1	1	1	0	
GA4	15	1	1	1	1	4

TG: SPRU FE ENGG

We now merge the possible Minterms from adjacent groups. The minterms which differ in only one bit position from adjacent groups are merged. The differed bit is represented by the symbol '-'. The remaining bits are kept as they are -

Group Name	Minterms	W	X	Y	Z	Comments
GB1	2, 6	0	-	1	0	Bits of x changes: w, x, z remains same ∴ x is denoted by '-'
	2, 10	-	0	1	0	Bits of w changes: x, y, z remains same ∴ w is denoted by '-'
	8, 9	1	0	0	-	Bits of z changes: w, x, y remains same ∴ z is denoted by '-'
	8, 10	1	0	-	0	Bits of y changes: w, x, z remains same ∴ y is denoted by '-'
GB2	6, 14	-	1	1	0	Bits of w changes: x, y, z remains same ∴ w is denoted by '-'
	9, 11	1	0	-	1	Bits of y changes: w, x, z remains same ∴ y is denoted by '-'
	10, 11	1	0	1	-	Bits of z changes: w, y, x remains same ∴ z is denoted by '-'
	10, 14	1	-	1	0	Bits of x changes: w, y, z remains same ∴ x is denoted by '-'
GB3	11, 15	1	-	1	1	Bits of x changes: w, y, z remains same ∴ x is denoted by '-'
	14, 15	1	1	1	-	Bits of z changes: w, x, y remains same ∴ z is denoted by '-'

TG: SPPU_FE_ENGG

Step III: we repeat Step II. In the following table, there is Merging of Minterm pairs from adjacent groups.

Group Name	Minterms	W	X	Y	Z	
GB1	2, 6, 10, 14	–	–	1	0	} repeated rows
	2, 10, 6, 14	–	–	1	0	
	8, 9, 10, 11	1	0	–	–	} repeated rows
	8, 10, 9, 11	1	0	–	–	
GB2	10, 11, 14, 15	1	–	1	0	} repeated rows
	10, 14, 11, 15	1	–	1	0	

Step IV:- In this case there are two groups & each group contains combinations of four min-term

These combinations of 4 minterms are repeated in two rows. So, we remove the repeated rows.

After removing the redundant rows, the table is as follows:-

Group Name	Minterms	W	X	Y	Z
GC_1	2, 6, 10, 14	–	–	1	0
	8, 9, 10, 11	1	0	–	–
GC_2	10, 11, 14, 15	1	–	1	–

Here Merging of the combinations of Minterms from adjacent groups is not possible, since they differed in more than one-bit position.

There are three rows in the above table so, each row will give one prime.

TG: SPPU_FE_ENGG

∴ The prime Implicants are $y\bar{z}$ [for 1, y & for 0, $\bar{z}$], $w\bar{x}$ & wy

∴ $f(w,x,y,z) = y\bar{z} + w\bar{x} + wy$ is Minimised Boolean Expression.

⇒ Data Representation:-

✯ Representation of data

Computer does not understand human language Any data e.g. letters, pictures etc. that to be fed to computer should be converted to Machine language. computer represents data in the following Form.

✯ Binary Addition.

Rules of Binary Addition.

A+B	sum	carry
0+0	0	0
0+1	1	0
1+0	1	0
1+1	0	1

Find 0011010 + 0001100

```
  11
  0011010      (26)10
  0001100      (12)10
  ------       ------
  100110       (38)10
```

✯ Binary Subtraction:-

Rules of Binary subtraction.

A-B	Sub	Borrow
0-0	0	0
1-0	1	0
1-1	0	0
0-1	1	1

TG:SPRU_EE_ENGG

Sign-Magnitude Representation:-

In decimal number system a plus (+) sign & (−) sign used to denote positive & negative number This representation of numbers is known as 'signed number'.

In Binary number system, to indicate the sign of numbers, an additional bit is used as the 'sign bit' & it is placed as the most significant bit A '0' is used to represent a 'positive number' & '1' to represent a 'Negative Number'.

e.g. an 8-bit Signed number -01000100.

represents a positive number & its value (Magnitude) is $(1000100)_2 = (68)_{10}$

The '1' in the left-Most position (MSB) indicates that the number is negative, with Magnitude $(1000100)_2 = (68)_{10}$

This kind of representation for signed representation is known as 'sign-Magnitude Representation'.

Find the decimal Equivalent of the following binary numbers assuming sign magnitude representation of the binary numbers.

(1) 101100 (2) 0111

TG: SPPU FE ENGG

Soln Left Most bit is 1 i.e. sign bit is 1, which Means the no. is negative.

Magnitude - 01100 $= (12)_{10}$

$\therefore$ 101100 $= (-12)_{10}$

$(1\ 0\ 11\ 00)_2$

↓ signed bit — Magnitude

Ex2 :- convert $(-124)_{10}$ to its equivalent sign Magnitude form.

Soln :-

Division by 2	Quotient	Remainder	
124	62	0	
62	31	0	
31	15	1	↑ MSB
15	07	1	
07	03	1	
03	01	1	
01	—	1	

$\therefore (124)_{10} = (1111100)_2$

For negative number, we will add a sign bit '1' to the left-Most bit of the number.

$(1\ 1111100) = (-124)_{10}$

↓ signed bit — ← Magnitude →

✱ One's complement Representation :-

In a binary number, if each 1 is replaced by 0 & each 0 by 1, the resulting number is known as the 'one's' complement.

In one of these numbers is positive, then the other number will be negative with the same magnitude & vice-versa.

TG:SPPU_FE_ENGG

e.g. $-(0101)_2$ represents $(+5)_{10}$ where as $(1010)_2$ represents $(-5)_{10}$

In this representation also, MSB is 0 for positive numbers & 1 for negative number.

EX:- Find the one's complement of 0100111001

Soln:- The complement is

1 0 1 1 0 0 0 1 1 0

(2) Represent the following numbers in one's complement form - ① -7 & -7 ② +15 & -15

Soln:-

$(+7)_{10} = (0111)_2$

$(-7)_{10} = (1000)_2$

$(+15)_{10} = (01111)_2$

$(-15)_{10} = (10000)_2$

Two's complement Representation:-

Two's complement is a Mathematical operation on binary numbers. It is an example of radix complement.

If 1 is added to 1's complement of a binary number, the resulting number is known as the two's complement of the binary number.

e.g. consider the number 0101

$\therefore$ 1st complement : 1010

$$\begin{array}{r} 1010 \\ +\quad\ 1 \\ \hline 1011 \end{array}$$

$\therefore$ 2's complement of 0101 is 1011

$\therefore (0101)_2 = (+5)_{10}$

$\therefore (1011)_2 = (-5)_{10}$ in 2's complement

Remark: The 2's complement of the 2's complement of a number is the number itself.

TG:SPRUFFENGG

Ex:- Find the 2's complement of the Number

① 01001110

Soln:- The given number is 01001110

1's complement : 10110001

adding 1

$$\begin{array}{r} 10110001 \\ + \quad\quad 1 \\ \hline 10110010 \end{array}$$

2's complement is = 10110010

Ex:- Represent $(-17)_{10}$ in

① sign Magnitude

② One's Complement

③ Two's complement

Soln:- The Minimum number of bits required to represent $(+17)_{10}$ in signed number format is six.

$\therefore (+17)_{10} = (010001)_2$

$\therefore (-17)_{10}$ is represented by 110001 in sign-Magnitude representation.

1's complement = 101110 (first bit 1 represent the number is -ve)

Adding 1 to 1's complement.

$$\begin{array}{r} 101110 \\ + \quad\quad 1 \\ \hline 101111 \end{array}$$

$\therefore$ 2's complement = 101111

TG-SPRUFFE_ENGG

✡ Binary Subtraction Using 1's Complement Method:-

The steps to subtract two binary numbers using 1'st complement -

(1) Find 1's complement of the subtracted (i.e. the term to be subtracted)

(2) Add the complement number with the (i.e. the term from which subtraction is Made)

(3) If we get the carry, add the carry to its LSB, & we get the result.

(4) If the MSB=0 then the result is positive & in true binary form.

If the MSB=1 then the result is negative, then take its complement to convert it to true binary form.

Thus, if there is no carry over, then 1's Complement of the result of addition is obtained to get the final result & it is negative.

Ex:- Find subtraction using 1's complement Method.

10101 - 00111

Soln:- Subtracted is 00111

We take its 1's complement: 11000

Now we sum them i.e.,

```
  1 0 1 0 1
+ 1 1 0 0 0
-----------
1 0 1 0 1 0
```

In the above result, we get the carry bit 1, so we add this to the LSB of the given Result.

i.e.

```
    1
  0 1 1 0 1
        1
-----------
  0 1 1 1 0
```

Ans: 01110

TG SPRUFE ENGG

Ex:- Find 10101 - 10111

Soln:- we take 1's complement of subtracted 10111

↓

01000

$$\begin{array}{r} 10101 \\ 01000 \\ \hline 11101 \end{array}$$

In this result we didn't get the carry bit, so we calculate the 1's complement of the result i.e. (00010). which is negative number & the final answer.

✗ Binary subtraction using 2's complement Method:-

In computers, subtraction of number is done using addition of one number with the 2's complement of the number.

e.g. = (X-Y) = X + (2's complement of Y)

1. First take 2's complement of second number
2. Add first number to the 2's complement of second number
3. If carry is generated then result number is positive and true form. Remove carry or ignore carry.
4. If carry is not generated then the result number is -ve and in the 2's complement form.

TG-SPPU-ENGG

EX:- Evaluate 110110 − 10110

Soln:- The number of bits in the standard is 5 while that of Minuend is 6. Hence we add a '0' in the sixth place of the Subtracted.

To find 2's complement of 010110

1st complement is : 101001

2's complement

$$\begin{array}{r} 101001 \\ +\quad\quad 1 \\ \hline 101010 \end{array}$$

We add this with the Minuend.

$$\begin{array}{rr} & 1\,1\,1\,1 \\ & 1\,1\,0\,1\,1\,0 \\ + & 1\,0\,1\,0\,1\,0 \\ \hline \boxed{1} & 1\,0\,0\,0\,0\,0 \end{array}$$

After dropping the carry over, we get the result of subtraction to the 100000.

TG: SPPU FE ENGG

Page No.:
Date: 29/07/24

Class Test (DIV-B)

Q-1. Explain any Four (8 M)

1. Perform binary subtraction using 1's complement Method.

   $(28)_{10} - (15)_{10}$.

2. Find the one's complement of the following numbers.

   10110001 & 00100011

3. Perform following operations using 2's complement Method.

   $(4)_{10} - (7)_{10}$

4. Represent $(-11)_{10}$ in
   (i) Sign Magnitude
   (ii) 1's complement
   (iii) 2's complement form.

5. perform binary addition for
   $(8)_{10}$ and $(12)_{10}$

Q-2. Explain any two [12M]

1. Simplify the following logic function using the Quine-McCluskey Minimization Technique.

   $Y(A, B, C, D) = m(0, 1, 3, 7, 8, 9, 11, 15)$

2. Explain 3 forms of signed binary number representation.

3. Minimize the following logic function using K-Map and realize using logic gates. Draw the logic diagram for the output.

   $F(A, B, C, D) = \Sigma m(1, 3, 7, 11, 15) + d(0, 2, 5)$

RG.SPPU_FE_ENGG

M T W T F S S
Page No.:
YOUVA
Date 30/07/24

Class Test (DIV-A)

Q-1. Explain any Four (8 M).

1. perform binary subtraction using 1's complement Method.
   $(12)_{10} - (8)_{10}$
2. Find the one's complement of the following numbers.
   00011001 and 10110001
3. perform following operations using 2's complement Method.
   $-(48)_{10} - (23)_{10}$
4. Represent $(-25)_{10}$ in
   i) sign Magnitude
   ii) 1's complement
   iii) 2's complement form.
5. perform binary addition for
   $(7)_{10}$ and $(5)_{10}$

Q-2. Explain any two [12 M]

1. Simplify the following function using Quine-Mcclukey Minimization Technique:
   $Y(A, B, C, D) = \Sigma m\,(0, 1, 2, 3, 5, 7, 8, 9, 11, 14)$.
2. What is SOP & POS form? How to represent given SOP equation in K-Map, Explain with example.
3. Minimize the following function using K-map and implement using basic logic gates.
   $f(A, B, C, D) = \Sigma m(1, 3, 5, 8, 9, 11, 15) + d(2, 13)$.

TO SPPU FE ENGG

Assignment :- 1

1. Explain 3 forms of Signed binary number representation.
2. Minimize the following logic function using K-Map and realize using logic gates. Draw the diagram for the output.
   $F(A,B,C,D) = \Sigma m(1,3,7,11,15) + d(0,2,5)$
3. Solve by Quine McClusky technique:
   $Z = f(A,B,C,D) = \Sigma m(0,1,3,4,6,8,10,12,14)$
4. perform the following operations using 2's complement Method:
   (i) $-(48)_{10} - (+23)_{10}$
   (ii) $-(48)_{10} - (-23)_{10}$
5. What are don't care conditions? Explain with Example.
6. Minimize the following logic function using K-Map and realize using logic gates:
7. $F(A,B,C,D) = \Sigma m(1,5,7,13,15) + d(0,6,12,14)$
8. Simplify the following logic function using the Quine-McCluskey Minimization technique.

TG: SPPU_FE_ENGG